

A verified CAD algorithm and decision procedure in PVS

An overview of the `pvs_cad` library and its proof strategies `(cad)` and `(cad-qe)`

J. Tanner Slagel*

Code and proofs: https://github.com/j-tanner-slagel/pvs_cad

In short. `pvs_cad` is a PVS library, with the proof strategies `(cad)` and `(cad-qe)`, that decides the first-order theory of the real numbers and eliminates quantifiers: formulas built from polynomial equations and inequalities with rational coefficients, the connectives AND, OR, NOT, IMPLIES and IFF, and quantifiers over real variables. Two executable PVS functions compute a cylindrical algebraic decomposition (CAD). For the cells built from Collins’s projection operator the library proves, for every family of polynomials in every number of variables and with no run-time certificate, that the cells of $\mathbb{R}^n$ cover it and are disjoint, that the cells of every level are connected and first-order definable and stack in cylinders over continuous, strictly ordered root functions with definable graphs, and that each cell of $\mathbb{R}^n$ carries one sign vector of the input polynomials (`col_cells`); `col_found` returns this CAD as data, at least one record with an exact sample point for every nonempty cell of $\mathbb{R}^n$ and none for any other, and those records decide every sentence over the polynomials in the same variables, in the same order. The n-level engine, built first, computes such a CAD in two or more variables whenever its run-time certificate holds, which some value of its search-effort parameter u always achieves (`cad_verified`). A set is definable by a first-order formula with rational coefficients exactly when a quantifier-free one defines it (`fod_qfd`, Tarski–Seidenberg over $\mathbb{Q}$), and every cell of both CADs, at every level, is quantifier-free definable (`cad_sa`). In a theory that imports `pvs_cad`, `(cad)` decides a closed prenex formula over `real` whose atoms compare polynomials in its bound variables by `decide8`, the Collins run (after a witness search), and any other first-order formula over the reals, one with other real terms included, by `decide_g`; without `cad_decide8` it uses `decide5`, built on the n-level engine. All three are proved *sound* (an answer is always the truth) and *complete* (they always answer). `(cad-qe)` replaces a formula with free variables by a quantifier-free one that holds at exactly the same points. Inside a proof, the strategies evaluate the procedure and turn its answer into a PVS proof that the kernel checks. The whole library, 355 theories and 5,183 formulas, is proved.

Using it. A theory needs one line, `IMPORTING pvs_cad`. Then, in a proof:

<code>(cad)</code>	decide the goal, a first-order sentence over the reals: a TRUE goal is proved; otherwise it stays, labelled <code>cad</code> , with a message saying it is FALSE
<code>(cad -1)</code> , <code>(cad 2)</code> , <code>(cad (-1 3))</code>	the same on a chosen hypothesis or goal (a FALSE hypothesis closes the goal), or on several formulas together
<code>(cad *)</code> , <code>(cad +)</code> , <code>(cad -)</code>	decide the sequent as a whole (the hypotheses imply the goals), or the goals or the hypotheses alone; other real terms, such as $f(c)$ or $\sqrt{c}$, become unknowns
<code>(cad-qe)</code> , <code>(cad-qe -1)</code>	replace a goal or hypothesis with free real terms by an equivalent quantifier-free formula
<code>(cad :cad-only? t)</code> , <code>(cad-direct)</code>	answer only through the verified CAD engine; decide without the witness search

Formulas are taken as written: grouped binders, quantifiers inside NOT, AND, OR, IMPLIES and IFF, PVS’s synonyms for the connectives, and binders over `posreal`, `nnreal`, `negreal`, `npreal` and

*This report was written with Claude Code using Claude Opus 5.5. The `pvs_cad` library was developed the same way, with Claude Fable 5.1 and then Opus 5 and Opus 5.5; every proof in it is checked by PVS. Claude Code: <https://claude.com/claude-code>.

nzreal. cad/cad_showcase.pvs has 43 commented examples, each with its proof and its time, and cad/cad_forms_ex.pvs 64 regression lemmas.

What it looks like

Real PVS sessions, recorded on 1 October 2026 in cad_showcase, which imports pvs_cad, with the prover's commentary and the simplifier's internal messages cut. The quadratic formula as an existence theorem, written with a grouped binder ((cad) proves it from its nested form; the timing table below gives the whole proof of that nested form, d_quad_suf):

```
quadratic_formula :
|-----
{1}  FORALL (a, b, c: real):
      EXISTS (x: real):
        a /= 0 AND b ^ 2 >= 4 * a * c IMPLIES a * x ^ 2 + b * x + c = 0
Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
Real time = 5.297 secs.
```

Continuity of x^2 at 0, the epsilon-delta way: three alternating quantifiers, binders over posreal and an implication inside, taken as written:

```
sq_continuous :
|-----
{1}  FORALL (e: posreal):
      EXISTS (d: posreal):
        FORALL (x: real): -d < x AND x < d IMPLIES x ^ 2 < e
Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
Real time = 1.81 secs.
```

(cad *) works on a whole sequent. After (skeep) the facts sit in the sequent over skolem constants; $f(x)$ and $\text{length}(l)$ become unknown reals, and that $\text{length}(l)$ is nonnegative comes from its type. The strategy says which formulas it uses, which terms become unknowns, and which sentence it decides:

```
unknown_terms :
|-----
{1}  FORALL (f: [real -> real], l: list[real], x: real):
      f(x) > x ^ 2 AND length(l) = 3 IMPLIES f(x) + length(l) > 2
Rule? (skeep)
{-1} f(x) > x ^ 2
{-2} length(l) = 3
|-----
{1}  f(x) + length(l) > 2
Rule? (cad *)
cad*: using (-1 -2 1), skipping nothing; unknown reals: cadv_1 = f(x), cadv_2 = x, cadv_3 = length(l)
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_3 >= 0))
  ↳ IMPLIES (((cadv_1 > (cadv_2 ^ 2)) AND (cadv_3 = (3))) IMPLIES (((cadv_1 + cadv_3) > (2)))))
Deciding * by cylindrical decision, witness first,
Q.E.D.
Real time = 2.292 secs.
```

Quantifier elimination inside a proof: (cad-qe -1) prints the quantifier-free answer and replaces the hypothesis by it, with a proof that the two are equivalent; (cad *) finishes:

```

qe_then_cad :
|-----
{1}  FORALL (a, b: real):
      (EXISTS (t: real): t ^ 2 + a * t + b = 0) AND b > 0 IMPLIES
      a ^ 2 >= 4 * b
Rule? (skeep)
{-1}  EXISTS (t: real): t ^ 2 + a * t + b = 0
{-2}  b > 0
|-----
{1}  a ^ 2 >= 4 * b
Rule? (cad-qe -1)
cad-qe: ((b < 0) OR ((b = 0) OR ((b > 0) AND ((4*b + (-1)*a^2 = 0) OR (4*b + (-1)*a^2 < 0)))))
Eliminating the quantifiers of -1,
{-1}  ((b < 0) OR
      ((b = 0) OR
      ((b > 0) AND
      ((4 * b + (-1) * a ^ 2 = 0) OR (4 * b + (-1) * a ^ 2 < 0)))))
{-2}  b > 0
|-----
{1}  a ^ 2 >= 4 * b
Rule? (cad *)
cad*: using (-1 -2 1), skipping nothing; unknown reals: cadv_1 = b, cadv_2 = a
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): (((cadv_1 < (0)) OR ((cadv_1 = (0)) OR
  ↳ ((cadv_1 > (0)) AND (((4) * cadv_1 + ((-1) * (cadv_2 ^ 2))) = (0)) OR (((4) * cadv_1 + ((-1) *
  ↳ (cadv_2 ^ 2))) < (0))))) AND (cadv_1 > (0))) IMPLIES (((cadv_2 ^ 2) >= ((4) * cadv_1)))
Deciding * by cylindrical decision, witness first,
Q.E.D.
Real time = 4.596 secs.

```

How a proof by (cad) works

The strategy follows the reflection template of Narkawicz, Muñoz and Dutle’s univariate Sturm and Tarski strategies in NASALib [4, 5]. (cad) reads the goal and computes its encoding as data (os, F, phi): the quantifier prefix, the polynomials’ coefficient lists and the Boolean skeleton (see “The main theorems”). It then runs the decision in PVS’s ground evaluator (decide8, the Collins run, in a theory that imports cad_decide8, as pvs_cad does; decide5 in one that imports cad_decide5 but not cad_decide8) and instantiates its correctness theorem (decide8_correct or decide5_correct) with that data. Unfolding fsem, the semantics, turns the result back into the original formula: each quantifier is matched, and one reflection lemma per polynomial (peval_pnorm) shows that the coefficient list evaluates to the polynomial as written. A FALSE sentence is reported and left in place, and a false hypothesis closes the goal. Before deciding a prenex sentence, (cad) looks for a witness: a model search proves a goal whose quantifiers are all existential (or refutes a hypothesis whose quantifiers are all universal) by instantiating a point, and with three or more quantifiers, when the decision is slow, one variable (existential in a goal, universal in a hypothesis being refuted) is fixed to a small constant and the smaller sentence decided. Grouped binders such as FORALL (x, y: real) are decided through the nested form, and a goal NOT A as the hypothesis A. In a theory that imports pvs_cad, a formula of any other shape (quantifiers inside connectives; binders over posreal, nnreal, negreal, npreal or nzreal) is encoded as a term g of a datatype that mirrors PVS’s connectives; (cad) evaluates decide_g(g), instantiates decide_g_ok, unfolds the semantics gsem, which then has the formula’s own shape, and closes the goal connective by connective. (cad-qe) does the same for a formula with free real terms (skolem constants, say): it evaluates qe8 (qe in a theory that does not import qe8), or qe_g for other shapes, and replaces the formula by the quantifier-free answer, proved equivalent from qe8_correct, qe_correct or qe_g_ok. (cad *) collects the polynomial hypotheses and goals of a whole sequent, replaces non-polynomial real terms such as sqrt(x) or f(x) by universally quantified reals, and decides the implication, which enters the proof by case, so both branches are checked by PVS.

The trusted base is the PVS kernel plus the ground evaluator that runs the procedure (`decide8`, `decide_g`, `qe8` or `qe_g` in a theory that imports `pvs_cad`; `decide5`, `decide7` or `qe` in one that imports only `cad_decide5`, `cad_decide7` and `qe_all`), the same base as NASALib’s Sturm and Tarski strategies. It also includes NASALib’s saved proofs of the theorems the library imports, which were not replayed here. Beyond that, one trusts the specification: that the semantics (`fsem`, `gsem`, `qfsem`) and the theorem statements (last section) say what is intended. The strategy code (3,371 lines of Lisp) is not trusted: it only chooses proof steps.

What it decides in practice

CAD is doubly exponential in the worst case, so completeness means “always answers, given time”. Measured times for complete proofs on one M-series Mac: with `decide5`, in theories that import `cad_decide5`, on 28 September 2026 (PVS’s own timing); with `IMPORTING pvs_cad` (`decide8`, the Collins run) on 1 October 2026 (wall clock of the proof command, through `pvs-cli`):

Sentence	Shape	<code>decide5</code>	<code>pvs_cad</code>
Wilkinson’s $(x-1) \cdots (x-20)$, expanded, is positive for $x < 1$ (coefficients to 2.4×10^{18})	$\forall$, degree 20	2.9 s	3.2 s
Chebyshev T_{10} stays in $[-1, 1]$ on $[-1, 1]$	$\forall$, degree 10	2.0 s	2.7 s
Motzkin’s polynomial is nonnegative (not a sum of squares [13])	$\forall\forall$, degree 6	1.1 s	1.7 s
Every depressed cubic $x^3 + px + q$ has a real root	$\forall\forall\exists$	0.8 s	1.5 s
Quadratic formula: $a \neq 0 \wedge b^2 \geq 4ac \Rightarrow \exists x : ax^2 + bx + c = 0$	$\forall\forall\forall\exists$	2.1 s	3.2 s
$\forall x \exists y \forall z : z^4 + yz^2 + y \geq x$	$\forall\exists\forall$	22.9 s	8.3 s
No real root of $z^2 + bz + c$ implies $b^2 < 4c$, by (<code>cad *</code>) on a sequent	quantified hypothesis	1.3 s	2.0 s

With `decide5` the limits show up with three or four variables at degree 3 to 4: AM–GM for three or four variables and Cauchy–Schwarz in the plane did not finish in two minutes (28 September 2026). With `pvs_cad` AM–GM for three variables takes 3.4 s and Cauchy–Schwarz in the plane 2.2 s, while AM–GM for four variables still does not finish in two minutes (1 October 2026). On 100 generated sentences with 1 to 6 quantifiers, (`cad`) with `decide5` proved 98 within 300 s, and no answer disagreed with Z3 (27 September 2026). On 68 (`cad`) and (`cad-qe`) proofs from the library’s example theories, proofs with the Collins imports (`decide8`, `qe8`) took 0.6–0.8 s longer on average than with `decide5` and `qe` (0.77 s in a first run, 0.59 s in a re-run with the strategy fixes of this release), partly because of the larger imported context; on some problems with several quantifiers the Collins decision was faster by more than an order of magnitude (23 to more than 450 times) and on others slower (1 October 2026; docs/collins_cad.pdf).

How it was built

The library was built in stages from 9 September to 2 October 2026 (298 commits). Most stages also shipped a proof strategy of their own.

1. **Multivariate polynomials.** A normal form for multivariate polynomials over the rationals, proved unique, and reflection lemmas connecting it to PVS real expressions (strategies `mpoly-eq`, `mpoly-simp`).
2. **Real algebraic numbers.** Root isolation, ordering and signs at algebraic numbers, on top of NASALib’s Sturm and Tarski libraries (`alg-roots`).
3. **Branching quantifier elimination.** One-variable quantifier elimination with parametric sign conditions (`qe-exists`, `poly-sign`).
4. **Resultants and subresultants:** determinants, the resultant, and the principal subresultant coefficients with their specialization property. Two projection operators built from them are defined but not proved delineating: a McCallum-shaped one (`cad_proj`), from which the decisions of item 6 start their read closures

(their soundness rests on run-time checks instead), and a Collins-like one without reducta (`cad_projc`). Collins's own operator, with reducta (the N-reducta projection), is proved delineating in item 10.

5. **A first cylindrical decision**, `decide`, proved correct for every sentence (`decide_correct`). It was complete but exponential in the number of polynomials: a circle and a line did not finish in 15 minutes.
6. **The fast cylindrical decision**, which `decide5` uses for two or more quantifiers; (`cad`) runs `decide5` in a theory that does not import `cad_decide8`. Over each cell of the lower variables, it lifts to the next variable by walking exact rational separators between that variable's roots, and reads the sign vector at each separator and root through Tarski queries at the cell's sample point, which may be algebraic. Soundness never assumes what it cannot check: the signs each lifting step reads are certified constant on each sector at run time, which is proved to fix the set of sign vectors in every fibre over the sector, and a failed check sets `ok` to `FALSE` (`decw_correct` for two quantifiers, `decn_correct` for any number).
7. **Completeness**. A review showed that the only way `ok` could be `FALSE` was a fixed search limit running out. Some searches were made total; the others share one fuel parameter u , and every large enough u makes every check pass: each search stops at its first success, so a larger u changes nothing once u suffices, and the closures that add sign reads draw them from a finite set built level by level from F , so they reach a fixpoint (`decn_complete`). `decn_u`, which `decide5` calls, raises u until the checks pass; it terminates on a non-computable measure, the distance to a bound beyond which every u suffices (`ubound`, chosen by `epsilon`), which keeps it an ordinary executable PVS function (`decn_u_complete`, then `decide5_decides`).
8. **The CAD**. The run-time checks give delineability in the sign-invariant sense (`delin_cl?`): if the walk's certificate holds at one point of a connected set and the signs it reads there hold across the set, the family's roots over the set are continuous and never cross, and the family has one sign vector on each section and band (`stack_const`). From this, the run's cells form a CAD (`cad_run`), `cad_out` returns them as data that decides every sentence over F in the same variables (`cad_out_ok`, `cad_decides`), and every cell and root function's graph is first-order definable (`rcell_fod_le`, `graph_fod`); `cad_verified` puts them together.
9. **Quantifier elimination**. For every prenex formula with at least one quantifier and $m \geq 1$ free variables, `qe` returns a quantifier-free formula that holds at exactly the same points of $\mathbb{R}^m$ (`qe_complete`). With two or more free variables, when the plain run's answer is not quantifier-free, `qe` falls back to a run whose free levels are closed under the derivative, where Thom's lemma separates the cells. (`cad-qe`) uses `qe` in a theory that does not import `qe8`.
10. **Collins's projection**. Collins's N-reducta projection (`projn`, executable) is proved delineating for every input, by the subresultant theorem and the continuity of complex roots in the coefficients (`collins_stack`, `projn_ok`). The cells built from its tower form a CAD adapted to the family in every number of variables, with no run-time certificate (`col_cells`), which `col_found` returns as data (`col_found_cad`). `decide8` decides on that tower, and `qe8` eliminates quantifiers on it or on one whose free levels are closed under the derivative (`decide8_decides`, `qe8_complete`).
11. **Tarski-Seidenberg**. `qelim` eliminates the quantifiers of every first-order formula from the inside out, each existential quantifier by `qe8`, or by `decide8` when no variable is free (`qelim_qf`, `qelim_ok`). So a set is first-order definable over $\mathbb{Q}$ exactly when it is quantifier-free definable (`fod_qfd`), and every cell of both CADs is (`cad_sa`).
12. **Formulas as written**. A datatype `Gm` mirrors PVS's connectives; `decide_g` decides its sentences and `qe_g` eliminates its quantifiers, both through `qelim` (`decide_g_ok`, `qe_g_ok`). With `IMPORTING pvs_cad`, the one import (`cad`) and (`cad-qe`) need, (`cad`), (`cad *`) and (`cad-qe`) take formulas as written: grouped binders, quantifiers inside NOT, AND, OR, IMPLIES and IFF, and binders over `posreal`, `nnreal`, `negreal`, `npreal` and `nzreal` (a binder over any other type, such as `nat`, is refused).

The library in numbers

PVS theories	355	Formulas proved, whole-library replay	5,183 of 5,183
Lines of PVS (358 files, with the datatypes)	26,864	lemmas and theorems	3,553
Lines of strategy code (Lisp)	3,371	TCCs (type-correctness obligations)	1,630
Commits, from 9 September 2026	298	Axioms added	none

It builds on NASALib’s Sturm and Tarski (univariate root counting) and on reals, structures, analysis, complex, mult_poly, matrices and interval_arith.

Where this sits

Tarski showed that the first-order theory of the real numbers is decidable [1], and Collins’s cylindrical algebraic decomposition [2] made the decision practical; this library builds on the PVS [3] libraries of NASALib [4], in particular the univariate Sturm and Tarski decision procedures of Narkawicz, Muñoz and Dutle [5]. Earlier quantifier elimination for the reals with machine-checked answers includes, by methods other than CAD, Cohen and Mahboubi’s in Coq [6], McLaughlin and Harrison’s proof-producing procedure in HOL Light [7], and a verified, complete multivariate procedure in Isabelle/HOL [8], exported to executable code, whose authors report that without virtual substitution it is impractical beyond the simplest univariate examples. Mahboubi implemented CAD in Coq without a completed correctness proof [9]. A formal correctness proof of Collins’s CAD, including the delineability of Collins’s projection, exists in Rocq [10], over an abstract real closed field; it is a correctness proof, not an executable procedure used as a tactic. Lean’s Mathlib [11] has, as far as we could find, no CAD formalization and no CAD-based decision tactic, and a Lean project’s roadmap from September 2026 lists CAD, delineability and Sturm theory among what Mathlib lacks [12]; Lean’s nonlinear tactics (`nlinarith`, `polyrith`) are heuristic or check certificates found by an external algebra system, and are not complete. To our knowledge (a survey of September 2026), `pvs_cad` is the first multivariate decision procedure of the CAD family that is executable, proof-producing, formally verified sound and complete, and usable as a tactic in an interactive prover. The theorems about both its CADs (for the `n`-level engine, whenever its certificate holds) state the conditions of Basu, Pollack and Roy’s Def. 5.1, in two respects only indirectly: the partition is stated for the cells of $\mathbb{R}^n$, and at lower levels it follows from the stack construction (`tcell_cover`, `tcell_disj`) without being restated; finiteness comes through the records of `cad_out` and `col_found`, at least one for every nonempty cell. What is proved, and what is not:

- “Semi-algebraic” holds in two senses. The CAD theorems state that every cell, at every level, and every root function’s graph over a cell is defined by a first-order formula whose atoms are sign conditions on polynomials with rational coefficients. `fod_qfd (qelim_ok.pvs)` proves that a set is defined by such a formula exactly when a Boolean combination of such sign conditions defines it without quantifiers (Tarski–Seidenberg over $\mathbb{Q}$), and `cad_sa.pvs` states this for every cell, at every level, of both CADs and for the Collins CAD’s root graphs (`col_cells_sa`, `cad_cells_sa`). Sets so defined over $\mathbb{Q}$ are semi-algebraic in Basu, Pollack and Roy’s sense, whose definition also allows real coefficients and so admits more sets ($\{\pi\}$, for one). No tool prints the quantifier-free definitions of the cells; the theorems say that they exist.
- For the `n`-level engine, delineability is proved for the polynomial families the run builds and checks. For Collins’s `N-reducta` projection it is proved for every input (`collins_stack`, `col_stack.pvs`); `projn_ok` ties it to the executable operator `projn`, whose tower `decide8` and `qe8` use. McCallum’s and Brown–McCallum’s smaller projections are not proved delineating (the `n`-level engine starts its read closure from a McCallum-shaped operator, `cad_proj`, but its correctness does not rest on it), and Lazard’s is not done. Delineability is in the sign-invariant sense (`delin_cl?`): root multiplicities are not tracked.
- The Collins CAD theorems cover every number of variables, one included (`col_cells`; in one variable the cells are the sectors of the real roots of F , `col_cad_line`, the sectors over which `decide8` and `decide5` decide one-variable sentences); the `n`-level engine’s theorems (`cad_verified`) are stated for two or more.

- The CAD theorems cover the n-level engine and the Collins run. (cad) also has shortcuts they do not cover: the witness search and, with decide5, the two-level method decw for two variables; their answers are proved correct too. (cad :cad-only? t) uses neither: it decides by decide8 in a theory that imports cad_decide8 and by decide7 (the n-level engine) otherwise, and by decide_g for formulas of other shapes.

The main theorems

A prenex sentence is encoded as data: os lists the quantifiers outermost first (TRUE for $\forall$, FALSE for $\exists$), F lists the polynomials (exact rational coefficients), and phi is the Boolean skeleton, whose atoms say “polynomial i has sign s ”. Its meaning is defined by recursion on the prefix, with the real quantifiers of PVS itself:

The semantics (cad_decide.pvs)

```
fsem(os, F, phi, ys): RECURSIVE bool =
  CASES os OF
    null: bfeval(phi, F, ys),
    cons(q, r): IF q THEN FORALL x: fsem(r, F, phi, cons(x, ys))
                ELSE EXISTS x: fsem(r, F, phi, cons(x, ys)) ENDIF
  ENDCASES
```

The decision procedures return a record: val, the verdict, and ok, a flag that is TRUE when the run-time checks passed. decide8 is the Collins run; decide5 is built on the n-level engine:

Soundness and completeness (cad_decide8.pvs; cad_decide5.pvs, complete_all.pvs)

```
decide8_correct: THEOREM decide8(reverse(os), F, phi)`ok IMPLIES
                  (decide8(reverse(os), F, phi)`val IFF fsem(os, F, phi, null))
decide8_decides: THEOREM decide8(reverse(os), F, phi)`val IFF fsem(os, F, phi, null)
decide5_correct: THEOREM decide5(reverse(os), F, phi)`ok IMPLIES
                  (decide5(reverse(os), F, phi)`val IFF fsem(os, F, phi, null))
decide5_complete: THEOREM decide5(reverse(os), F, phi)`ok
decide5_decides: THEOREM decide5(reverse(os), F, phi)`val IFF fsem(os, F, phi, null)
```

There are no hypotheses: os, F and phi are arbitrary, so the theorems cover every closed prenex sentence with any number of quantifiers. Both procedures are total functions, and PVS generated and proved their termination obligations, so each always terminates and returns the truth of the sentence. (cad :cad-only? t) uses decide8 in a theory that imports cad_decide8, otherwise decide7 (cad_decide7.pvs), whose decide7_correct and decide7_decides have the statements above with decide7 in place of decide5.

Quantifiers anywhere need another formula type, rcf_fol's Fm (ftrue, sign atoms fsg(p, s), fnot, fand, fex; variables are positions in a point). Its semantics fsem(phi)(pt) is a different function from fsem(os, F, phi, ys) above; qfsem is the same semantics under another name, and qf?(phi) says that phi has no fex. gform_def's Gm mirrors PVS's connectives (TRUE, FALSE, the six comparisons of two polynomials, NOT, AND, OR, IMPLIES, IFF, FORALL, EXISTS) and its meaning gsem(g)(pt) is written with them; decide_g(g) and qe_g(g, m) apply qelim to its translation into Fm. qelim works from the inside out: each fex by qe8, or by decide8 when no variable is free.

In the theorems below on g and qelim the formula is arbitrary and m, where it occurs, is any natural number (m = 0: a sentence). In qe8_complete, m is a positive natural number and cons?(os) asks for at least one quantifier: every prenex formula with at least one quantifier and $m \geq 1$ free variables; qe_complete (qe_all.pvs) states the same for qe, built on the n-level engine. The atoms' polynomials have rational coefficients and the formulas have no parameters, so fod_qfd is the Tarski–Seidenberg theorem for formulas over $\mathbb{Q}$.

Any shape, quantifier elimination, Tarski–Seidenberg (gform_ok, qe8, qelim_ok, rcf_fol)

```

decide_g_ok: THEOREM decide_g(g) IFF gsem(g)(null)
qe_g_qf: LEMMA qf?(qe_g(g, m))
qe_g_ok: THEOREM length(pt) = m IMPLIES (fsem(qe_g(g, m))(pt) IFF gsem(g)(pt))
qe8_complete: THEOREM cons?(os) AND length(pt) = m IMPLIES
  qf?(qe8(m, os, F, phi)`out) AND (qfsem(qe8(m, os, F, phi)`out)(pt) IFF fsem(os, F, phi, pt))
qelim_qf: LEMMA FORALL phi: FORALL m: qf?(qelim(phi, m))
qelim_ok: THEOREM FORALL phi: FORALL m, pt: length(pt) = m
  IMPLIES (fsem(qelim(phi, m))(pt) IFF fsem(phi)(pt))
fod?(n)(S): bool = EXISTS phi: FORALL pt: length(pt) = n IMPLIES (S(pt) IFF fsem(phi)(pt))
qfd?(n)(S): bool = EXISTS phi: qf?(phi) AND FORALL pt: length(pt) = n IMPLIES (S(pt) IFF fsem(phi)(pt))
fod_qfd: THEOREM fod?(n)(S) IFF qfd?(n)(S)

```

The Collins CAD (col_stack.pvs, col_line.pvs, col_found_ok.pvs, cad_sa.pvs)

```

collins_stack: THEOREM conn?(C) AND C(p0) AND colh?(G, C) IMPLIES delin_cl?(G, C)
col_cells: THEOREM ccells?(k, F)
col_found_cad: THEOREM cons?(qs) IMPLIES cfound?(qs, F, col_found(qs, F)`freces)
col_cells_sa: THEOREM
  (FORALL s, A: member(s, csects(k, F)) AND length(A) <= k
    IMPLIES qfd?(1 + length(A))(ccell(k, F)(s, A)))
  AND (FORALL s, A, i: member(s, csects(k, F)) AND length(A) < k
    AND (FORALL q: ccell(k, F)(s, A)(q) IMPLIES i < nroot(nth(ctw(F, k), length(A)), q))
    IMPLIES qfd?(2 + length(A))
    (LAMBDA p: cons?(p) AND ccell(k, F)(s, A)(cdr(p))
      AND car(p) = rt(nth(ctw(F, k), length(A)), cdr(p), i)))

```

`colh?(G, C)` says that Collins's projection of G has one sign on C : the leading coefficients of the kept reducta, their principal subresultant coefficients with their derivatives, and those of pairs of members; `projn_ok` (`cad_projn.pvs`) derives it from the sign-invariance of the executable operator `projn`. `ccells?(k, F)` states, for the cells `ccell(k, F)(s, A)` built from the tower `ctw(F, k)` of Collins projections of a family F in $k + 1$ variables, that the cells of $\mathbb{R}^{k+1}$ cover it and are pairwise disjoint, that every cell at every level is connected and first-order definable, that over each sector of the line every level is delineable over every cell below it, that the graph of every root function over a cell is definable, and that F has one sign vector on every cell of $\mathbb{R}^{k+1}$; `col_cells` has no hypothesis: the cells form a CAD with no run-time check (`col_found` and `decide8` still check their walk's certificate while they raise u , which always ends, `decc_complete`). `cfound?(qs, F, L)` adds the records L : each lies in its cell, with an exact sample point and F 's signs there, and every nonempty cell of $\mathbb{R}^n$, $n = \text{length}(qs)$, has at least one; by `col_found_decides` they decide every sentence over F in the same variables, in the same order. `col_cells_sa` states that every cell, at every level, and every root function's graph over a cell is quantifier-free definable.

The n-level engine's CAD (cad_verified.pvs, cad_sa.pvs)

```

cad_verified: THEOREM decn_o(u, qs, F, Psi)`ok AND length(qs) >= 2 IMPLIES cad_of?(u, qs, F)
cad_exists: THEOREM length(qs) >= 2 IMPLIES EXISTS u: cad_of?(u, qs, F)
cad_cells_sa: THEOREM cad_of?(u, qs, F) AND member(s, runsects(u, length(cdr(qs)), F))
  AND length(A) <= length(cdr(qs))
  IMPLIES qfd?(1 + length(A))(rcell(u, length(cdr(qs)), F)(s, A))

```

`cad_of?(u, qs, F)` states the same conditions as `ccells?` for the cells `rcell(u, k, F)(s, A)` of the n -level engine `decn_o`, where $k + 1 = \text{length}(qs)$ and each sector of the line has a tower of its own, with the records of `cad_out`: at least one, with a sample point and F 's signs, for every nonempty cell of $\mathbb{R}^n$, and none for any other. `cad_cells_sa`: whenever `cad_of?` holds, every cell, at every level, is quantifier-free definable.

References

- [1] A. Tarski. *A Decision Method for Elementary Algebra and Geometry*. RAND Corporation, 1948; 2nd ed., University of California Press, 1951.
- [2] G. E. Collins. Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In *Automata Theory and Formal Languages*, LNCS 33, pages 134–183. Springer, 1975.
- [3] S. Owre, J. M. Rushby, N. Shankar. PVS: A prototype verification system. In *CADE-11*, LNAI 607. Springer, 1992. <https://pvs.csl.sri.com>
- [4] NASALib, the NASA PVS library collection. <https://github.com/nasa/pvslib>
- [5] A. Narkawicz, C. Muñoz, A. Dutle. Formally-verified decision procedures for univariate polynomial computation based on Sturm’s and Tarski’s theorems. *Journal of Automated Reasoning* 54:285–326, 2015. <https://doi.org/10.1007/s10817-015-9320-x>
- [6] C. Cohen, A. Mahboubi. Formal proofs in real algebraic geometry: from ordered fields to quantifier elimination. *Logical Methods in Computer Science* 8(1:02), 2012.
- [7] S. McLaughlin, J. Harrison. A proof-producing decision procedure for real arithmetic. In *CADE-20*, LNCS 3632, pages 295–314. Springer, 2005.
- [8] K. Kosaian, Y. K. Tan, A. Platzer. A first complete algorithm for real quantifier elimination in Isabelle/HOL. In *CPP 2023*. ACM, 2023. <https://arxiv.org/abs/2209.10978>
- [9] A. Mahboubi. Implementing the cylindrical algebraic decomposition within the Coq system. *Mathematical Structures in Computer Science* 17(1), 2007.
- [10] Q. Vermande. Cylindrical algebraic decomposition in Coq/Rocq. In *CPP 2026*, pages 45–58. ACM, 2026. <https://doi.org/10.1145/3779031.3779100>
- [11] The mathlib Community. The Lean mathematical library. In *CPP 2020*. ACM, 2020.
- [12] Hex project (Lean), roadmap for the real algebraic geometry behind CAD, issue 10300, September 2026. <https://github.com/kim-em/hex-dev/issues/10300>
- [13] T. S. Motzkin. The arithmetic-geometric inequality. In O. Shisha (ed.), *Inequalities*, pages 205–224. Academic Press, 1967.